

Calculo Numérico - Estabilidade e overflow

September 2, 2026



1 Objetivos

Investigar a relação entre estabilidade numérica, passo de tempo e ocorrência de *overflow* em uma simulação simplificada de dinâmica dos fluidos computacional.

Ao final da atividade, o aluno deverá ser capaz de:

- explicar o overflow em números de ponto flutuante;
- distinguir instabilidade numérica de overflow;
- calcular o limite de estabilidade do método utilizado;
- detectar falhas numéricas em Python;
- avaliar os efeitos do tipo numérico.

2 Descrição do problema - Elaborado para o depois

Uma equipe está simulando o movimento de um fluido entre duas placas paralelas, separadas por uma distância de $H = 0,01$ m.

Inicialmente, o fluido está parado. No instante inicial, a placa localizada em $y = 0$ começa a se mover com velocidade constante $U = 1$ m/s, enquanto a placa em $y = H$ permanece fixa.

A viscosidade transmite gradualmente o movimento da placa para o interior do fluido. Contudo, em algumas configurações numéricas, o programa calcula velocidades crescentes e não físicas, até que ocorra overflow.

Desafio: reproduzir a falha, identificar sua causa e testar uma configuração numericamente estável.

3 Fundamentação teórica

3.1 Representação em ponto flutuante e overflow

Os valores de velocidade são armazenados em números de ponto flutuante, cuja representação pode ser expressa, de forma simplificada, por

$$x = (-1)^s m 2^e,$$

em que s representa o sinal, m o significando e e o expoente.

Como o número de bits é limitado, existe um maior valor finito representável em cada tipo numérico.

Tipo numérico	Maior valor finito aproximado
float32	$3,4 \times 10^{38}$
float64	$1,8 \times 10^{308}$

O **overflow** ocorre quando o resultado de uma operação ultrapassa a faixa representável pelo tipo numérico.

Em operações de ponto flutuante no NumPy, isso pode produzir valores infinitos e avisos de execução. Também é possível configurar o programa para interromper o cálculo ao detectar a falha.

Atenção: instabilidade numérica e overflow não são sinônimos. A instabilidade pode amplificar perturbações e erros; o overflow ocorre posteriormente, se alguma operação exceder a capacidade de representação. Uma solução pode estar incorreta muito antes de apresentar overflow.

3.2 Modelo físico

Considere um fluido newtoniano, com viscosidade constante, escoamento unidirecional, ausência de gradiente de pressão e ausência de variação ao longo das placas.

Nessas condições, a equação da velocidade é

$$\frac{\partial u}{\partial t} = \nu \frac{\partial^2 u}{\partial y^2},$$

em que:

- $u(y, t)$ é a velocidade paralela às placas, em m/s;
- y é a posição transversal, em m;
- t é o tempo, em s;
- ν é a viscosidade cinemática, em m^2/s .

As condições de contorno, para $t > 0$, são

$$u(0, t) = U, \quad u(H, t) = 0.$$

No interior do domínio, a condição inicial é

$$u(y, 0) = 0, \quad 0 < y < H.$$

3.3 Discretização

Utilizando Euler explícito no tempo e diferenças finitas centrais no espaço:

$$\frac{u_i^{n+1} - u_i^n}{\Delta t} = \nu \frac{u_{i+1}^n - 2u_i^n + u_{i-1}^n}{\Delta y^2}.$$

Isolando a velocidade no próximo instante:

$$u_i^{n+1} = u_i^n + r (u_{i+1}^n - 2u_i^n + u_{i-1}^n)$$

com

$$r = \frac{\nu \Delta t}{\Delta y^2}.$$

O índice i identifica a posição na malha e o índice n identifica o instante de tempo.

3.4 Condição de estabilidade

Para esse esquema de difusão unidimensional:

$$r \leq \frac{1}{2}.$$

Consequentemente:

$$\Delta t \leq \frac{\Delta y^2}{2\nu}$$

Uma interpretação simples é obtida reorganizando a atualização:

$$u_i^{n+1} = r u_{i-1}^n + (1 - 2r) u_i^n + r u_{i+1}^n.$$

Quando $0 \leq r \leq 1/2$, os coeficientes são não negativos e somam 1. A atualização funciona como uma média ponderada.

Quando $r > 1/2$, o coeficiente central torna-se negativo, permitindo oscilações que podem crescer e levar a overflow.

Observação: esse limite se aplica ao esquema utilizado nesta atividade. Outros métodos e equações de CFD possuem condições de estabilidade diferentes.

4 Dados da simulação

Xc	
Parâmetro	Valor
Distância entre placas, H	0,01 m
Viscosidade cinemática, ν	$10^{-4} \text{ m}^2/\text{s}$
Número de pontos, incluindo as paredes	21
Espaçamento, $\Delta y = H/(21 - 1)$	0,0005 m
Velocidade da placa móvel, U	1 m/s
Velocidade da placa fixa	0 m/s

Para esses dados:

$$\Delta t_{\max} = \frac{(0,0005)^2}{2 \times 10^{-4}} = 0,00125 \text{ s}.$$

Serão comparados:

- **Caso A:** $\Delta t = 0,001 \text{ s}$, com $r = 0,4$;
- **Caso B:** $\Delta t = 0,002 \text{ s}$, com $r = 0,8$.

5 Implementação no Google Colab

Execute o código a seguir. A função registra a maior velocidade em módulo a cada passo e interrompe a simulação quando detecta overflow ou operação inválida.

Os comentários do código foram escritos sem acentos para facilitar a compilação do documento com o pacote listings.

```
import numpy as np
import matplotlib.pyplot as plt

def simular(dt, tipo=np.float32, passos=2000, pontos=21):
    """Simula difusao de velocidade e detecta falhas."""

    H = 0.01
    nu = 1e-4

    y = np.linspace(0, H, pontos)
    dy = H / (pontos - 1)

    # Coeficientes no tipo numerico escolhido.
    r = tipo(nu * dt / dy**2)
    dois = tipo(2)

    # Fluido parado e placa em y=0 em movimento.
    u = np.zeros(pontos, dtype=tipo)
    u[0] = tipo(1)

    tempos = [0.0]
    maximos = [float(np.max(np.abs(u)))]
    falha = None

    # Transforma avisos numericos em excecoes.
    with np.errstate(over="raise", invalid="raise"):
        for passo in range(1, passos + 1):
            try:
                novo = u.copy()

                # Atualiza o interior e preserva as paredes.
                novo[1:-1] = (
                    u[1:-1]
                    + r * (
                        u[2:] - dois * u[1:-1] + u[:-2]
                    )
                )

            except FloatingPointError as erro:
                falha = passo
                print(
                    f"dt={dt}, {tipo.__name__}: "
                    f"falha no passo {passo}, "
                    f"t={passo * dt:.4f} s - {erro}"
                )
                break

            u = novo
            tempos.append(passo * dt)
            maximos.append(float(np.max(np.abs(u))))

    if falha is None:
```

```

    print(
        f"dt={dt}, {tipo.__name__}: "
        f"{passos} passos sem overflow."
    )

    return {
        "y": y,
        "u": u,
        "tempos": tempos,
        "maximos": maximos,
        "r": float(r),
        "falha": falha,
    }

print("Maior float32:", np.finfo(np.float32).max)
print("Maior float64:", np.finfo(np.float64).max)

resultados = {}

for tipo in (np.float32, np.float64):
    for dt in (0.001, 0.002):
        nome = f"{tipo.__name__}, dt={dt}"
        resultados[nome] = simular(dt, tipo)

plt.figure(figsize=(9, 5))

for nome, resultado in resultados.items():
    plt.semilogy(
        resultado["tempos"],
        resultado["maximos"],
        label=nome
    )

plt.xlabel("Tempo simulado (s)")
plt.ylabel("Maximo de |u| (m/s) - escala logaritmica")
plt.title("Crescimento da velocidade e overflow")
plt.grid(True)
plt.legend()
plt.show()

```

Neste problema, a velocidade física deve permanecer no intervalo

$$0 \leq u(y, t) \leq U.$$

Portanto, velocidades muito superiores a 1 m/s já indicam uma solução incorreta, mesmo que não tenha ocorrido overflow.

6 Atividades propostas

1. Verificação da estabilidade.

Calcule Δy , $\Delta t_{\max}$ e r para os casos A e B. Classifique cada caso como estável ou instável antes de executar o programa.

2. Detecção de overflow.

Execute os quatro testes, combinando os dois passos de tempo com os tipos `float32` e `float64`.

Construa uma tabela contendo: tipo numérico, passo de tempo, valor de r , ocorrência de overflow e iteração da falha, quando houver.

3. Interpretação física.

Analise o gráfico da maior velocidade em módulo ao longo do tempo. Explique por que o crescimento observado no caso instável não representa o comportamento físico esperado.

4. Capacidade de representação.

Compare os resultados com `float32` e `float64`. A mudança de tipo numérico elimina a instabilidade? Justifique a diferença no número de passos até a falha.

5. Refinamento da malha.

Utilize 41 pontos, passando `pontos=41` para a função, e mantenha inicialmente $\Delta t = 0,001$ s.

Calcule o novo espaçamento e o novo limite de estabilidade. Execute a simulação e explique o resultado.

Em seguida, escolha um passo de tempo abaixo do novo limite e repita o teste.

6. Verificação do perfil de velocidade.

Para uma execução estável e suficientemente longa, represente graficamente o perfil final de velocidade e compare com a solução estacionária:

$$u_{\text{est}}(y) = U \left(1 - \frac{y}{H} \right).$$

Essa expressão descreve o regime estacionário, e não todo o transiente.

Calcule a diferença máxima em relação ao perfil estacionário:

$$E_{\infty} = \max_i |u_i - u_{\text{est}}(y_i)|.$$

Discuta se o tempo simulado foi suficiente para aproximar o regime estacionário. Essa diferença pode incluir o efeito de um transiente ainda não concluído.

7. Conclusão.

Explique a relação entre refinamento da malha, passo de tempo, instabilidade numérica e overflow. Indique qual alteração corrigiu a causa da falha nesta simulação.

7 Entrega

Entregar um notebook do Google Colab contendo:

- os códigos executados;
- a tabela dos testes;
- o gráfico da velocidade;
- a comparação com o perfil estacionário;
- as respostas e a conclusão.